



Instituto de Ciencias Básicas e Ingeniería

**UNIVERSIDAD AUTÓNOMA
DEL ESTADO DE HIDALGO**

Licenciatura en Ciencias
Computacionales
Web Programming

Arreglos en JavaScript

Carlos Alberto Lara Hernandez
Semestre: 7 Grupo: 1
Curso: Enero - Julio
2025

Arreglos en JavaScript

Los **arreglos** (o *arrays*) en JavaScript son colecciones ordenadas de valores que se almacenan bajo un mismo nombre. MDN describe a un arreglo como un “objeto tipo lista” que contiene múltiples valores[1], e indica que cada elemento está ubicado en una posición numérica (índice) empezando en 0[2]. Esto los hace muy flexibles: un solo arreglo puede contener números, cadenas, objetos e incluso otros arreglos[3]. Por ejemplo, para imprimir una factura con decenas de productos basta con poner los nombres y precios en un arreglo y recorrerlo en un bucle, en lugar de usar variables separadas para cada ítem[1].

Propiedades y métodos principales de los arreglos

Los arreglos en JavaScript tienen diversas propiedades y métodos integrados para manipular sus elementos. Por ejemplo, la propiedad `.length` indica cuántos elementos hay en el arreglo[4]. Entre los métodos más usados están:

- **push(elemento) y pop():** `push()` agrega uno o más elementos al final del arreglo, mientras que `pop()` elimina y devuelve el último elemento[5][6].
- **unshift(elemento) y shift():** Son análogos a `push/pop` pero actúan al inicio del arreglo. `unshift()` inserta elementos al principio, y `shift()` remueve el primer elemento[5][6].
- **map(función):** Crea un nuevo arreglo aplicando una función a cada elemento del arreglo original. Por ejemplo, `arr.map(x => x*2)` dobla todos los valores. MDN explica que `map()` genera un *nuevo* arreglo con los resultados de la función aplicada a cada elemento[7].
- **filter(función):** Devuelve un nuevo arreglo con todos los elementos que cumplen cierta condición (la función devuelve `true`). Según MDN, `filter()` “crea un nuevo array con todos los elementos que cumplan la condición implementada por la función dada”[8]. Por ejemplo, `arr.filter(x => x%2===0)` retorna solo los números pares.
- **Otros métodos útiles:** Existen además `forEach()`, `reduce()`, `slice()`, `splice()`, etc. De éstos, `forEach()` ejecuta una función sobre cada elemento (sin devolver un arreglo) y `reduce()` acumula valores. También hay métodos para unir y separar cadenas, convertir arreglos a strings con `join()` o `toString()`, etc. En general es buena idea usar estos métodos integrados en lugar de escribir bucles manuales por claridad y eficiencia[7][8].

Por conveniencia, los arreglos se suelen crear con la sintaxis literal `[...]`. Como señala GeeksforGeeks, usar la literal es más legible y eficiente que usar `new Array()`[9]. Por ejemplo:

```
const a = [1, 2, 3]; // arreglo con 3 elementos
```

Los arreglos soportan cualquier tipo de dato: números, cadenas, objetos, otros arreglos, etc.

Objeto Math en JavaScript

El objeto global Math en JavaScript provee propiedades y métodos para constantes y funciones matemáticas. MDN lo define como “un objeto incorporado que tiene propiedades y métodos para constantes y funciones matemáticas”[\[10\]](#). A diferencia de otros objetos, Math no es un constructor (no se usa con new) y todas sus propiedades y métodos son estáticos[\[11\]](#). Por ejemplo, las constantes como Math.PI (π) o Math.E están predefinidas, y se pueden llamar funciones como Math.sin(x), Math.sqrt(x), etc.

El propósito del objeto Math es ofrecer métodos para operaciones comunes. Permite realizar cálculos sin tener que reinventar funciones matemáticas. Por ejemplo, en lugar de implementar nuestro propio algoritmo de potencia o redondeo, podemos usar Math.pow() o Math.floor(), lo cual simplifica el código y garantiza consistencia. Vale notar que muchas funciones de Math trabajan con números de tipo Number (no sirven con BigInt).

Métodos principales del objeto Math

Entre los métodos más relevantes de Math se encuentran:

- **Math.random():** Genera un número pseudoaleatorio en el rango [0, 1). Es decir, devuelve un flotante ≥ 0 y < 1 [\[12\]](#). Es útil para aleatorizar comportamientos, pero no es criptográficamente seguro. Por ejemplo, Math.random() * 10 da un número entre 0 y 10 (excluyendo 10).
- **Math.floor(x):** Redondea hacia abajo al entero más grande menor o igual que x. Por ejemplo, Math.floor(3.7) retorna 3. Como indica MDN, Math.floor() “siempre redondea hacia abajo y devuelve el mayor entero menor o igual que el número dado”[\[13\]](#).
- **Math.pow(base, exponente):** Calcula la potencia de un número. Devuelve base elevado a la exponente[\[14\]](#). Por ejemplo, Math.pow(2, 3) da 8. Este método es equivalente al operador exponent (en ES2016 también existe **).
- **Otros métodos:** Existen muchos más, como Math.ceil(x) (redondeo hacia arriba), Math.round(x) (redondeo al entero más cercano), Math.min(), Math.max(), Math.sqrt(x), y funciones trigonométricas (Math.sin, Math.cos, etc.). Estos cubren la mayoría de operaciones matemáticas básicas en JavaScript.

Ejemplos prácticos de uso

A continuación se muestran algunos ejemplos de uso de arreglos y funciones Math en JavaScript:

Uso de arreglos con métodos funcionales:


```
const numeros = [1, 2, 3, 4];
// Usar map() para transformar elementos
const dobles = numeros.map(n => n * 2);
console.log(dobles); // [2, 4, 6, 8]

// Usar filter() para filtrar elementos según condición
const pares = numeros.filter(n => n % 2 === 0);
console.log(pares); // [2, 4]
```

En este ejemplo, `map` y `filter` producen nuevos arreglos sin modificar `numeros`.

Uso del objeto Math:

```
// Número aleatorio entre 0 (inclusive) y 1 (exclusivo)
const aleatorio = Math.random();
console.log(aleatorio);

// Redondeo de números
console.log(Math.floor(3.8)); // 3
console.log(Math.ceil(3.2)); // 4

// Potencia
console.log(Math.pow(2, 5)); // 32

// Generar un entero aleatorio entre 0 y 9
const entero = Math.floor(Math.random() * 10);
console.log(entero); // valor entre 0 y 9
```

Estos ejemplos muestran usos típicos: crear nuevos arreglos transformados o filtrados, y usar funciones de `Math` para cálculos numéricos.

Buenas prácticas

- **Declaración de arreglos:** Definir arreglos con `const` o `let` usando la sintaxis literal `[...]`[\[9\]](#). Evitar `new Array()` salvo casos específicos. Esto mejora la legibilidad y el rendimiento[\[9\]](#).
- **No mutar el arreglo original sin necesidad:** Métodos como `map()` y `filter()` retornan nuevos arreglos en lugar de modificar el original[\[7\]](#)[\[8\]](#). Esto se alinea con buenas prácticas de programación funcional (inmutabilidad) y evita efectos secundarios inesperados.
- **Uso de métodos integrados:** Prefiere los métodos nativos (`forEach`, `map`, `filter`, `reduce`, etc.) en lugar de bucles manuales cuando sea posible. Esto hace el código más conciso y claro[\[7\]](#)[\[8\]](#).
- **Validación:** Antes de operar, verificar que la variable es un arreglo con `Array.isArray()`. Así se evitan errores cuando recibimos datos de fuentes externas.

- **Manejo de Math:** Conocer el comportamiento de los métodos de Math. Por ejemplo, para trabajar con redondeos es preferible usar `Math.round`, `Math.floor`, `Math.ceil` según corresponda, y para evitar errores de precisión usar `toFixed()` si necesitamos un número fijo de decimales. Además, recordar que `Math.random()` no es adecuado para seguridad criptográfica o juegos donde se requiera alta calidad aleatoria.
- **Nombres descriptivos y comentarios:** Asignar nombres claros a arreglos y funciones. Si se realizan cálculos matemáticos complejos (p.ej. conversiones de unidades, fórmulas), agregar comentarios o documentar su origen. Esto mejora la mantenibilidad y legibilidad del código.

Referencias

- Mozilla Developer Network. (s.f.). *Arreglos*. MDN Web Docs (español). Recuperado de [https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting/Arrays\[1\]](https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting/Arrays[1]).
- GeeksforGeeks. (2025, 3 de octubre). *JavaScript Arrays*. Recuperado de [https://www.geeksforgeeks.org/javascript/javascript-arrays/\[2\]\[9\]](https://www.geeksforgeeks.org/javascript/javascript-arrays/[2][9]).
- MDN Web Docs. (s.f.). *Array.prototype.map()*. Recuperado de [https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/map\[7\]](https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/map[7]).
- MDN Web Docs. (s.f.). *Array.prototype.filter()*. Recuperado de [https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter\[8\]](https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter[8]).
- MDN Web Docs. (s.f.). *Math*. Recuperado de [https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math\[10\]](https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math[10]).
- MDN Web Docs. (s.f.). *Math.floor()*. Recuperado de [https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math/floor\[13\]](https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math/floor[13]).
- GeeksforGeeks. (2024, 15 de julio). *JavaScript Math.random() Method*. Recuperado de [https://www.geeksforgeeks.org/javascript/javascript-math-random-method/\[12\]](https://www.geeksforgeeks.org/javascript/javascript-math-random-method/[12]).
- GeeksforGeeks. (2024, 15 de julio). *JavaScript Math.pow() Method*. Recuperado de [https://www.geeksforgeeks.org/javascript/javascript-math-pow-method/\[14\]](https://www.geeksforgeeks.org/javascript/javascript-math-pow-method/[14]).

[1] Arreglos - Aprende desarrollo web | MDN

https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting/Arrays

[2] [3] [4] [5] [6] [9] JavaScript Arrays - GeeksforGeeks

<https://www.geeksforgeeks.org/javascript/javascript-arrays/>

[7] `Array.prototype.map()` - JavaScript | MDN

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/map

[8] `Array.prototype.filter()` - JavaScript | MDN

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter

[10] [11] `Math` - JavaScript | MDN

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math

[12] JavaScript `Math.random()` Method - GeeksforGeeks

<https://www.geeksforgeeks.org/javascript/javascript-math-random-method/>

[13] `Math.floor()` - JavaScript | MDN

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/floor

[14] JavaScript `Math.pow()` Method - GeeksforGeeks

<https://www.geeksforgeeks.org/javascript/javascript-math-pow-method/>